

Introduction

In discussing the varimax in the assignment 3, the algorithm proposed by ChatGPT and utilized by sklearn looks daunting. In this note I will try to summarise what I have learn from ChatGPT, also include some detailed derivations which weren't explicitly given by it.

Two important questions

1. In optomizing problems, we clearly need the gradient of the objective function with respect to the argument, namely

$$\nabla_{\mathbf{R}} L(\mathbf{R}; \mathbf{A}) \equiv \mathbf{P}(\mathbf{R}; \mathbf{A})$$

But the algorithm doesn't look like gradient descent, it computes the svd of $\mathbf{P}$ instead, why?

2. How do one derive the formidable matrix representation of $\mathbf{P}(\mathbf{R}; \mathbf{A})$?

Starting from question 2

As stated in question 1, it's always useful to compute the gradient term for the optimization problem, namely

$$L = \sum_{j=1}^K \left\{ \sum_{i=1}^n (\tilde{A}_{ij})^4 - \frac{1}{n} \left(\sum_{i=1}^n \tilde{A}_{ij}^2 \right)^2 \right\}$$

Since the argument of L is R , which is a matrix, the gradient of L should also be a matrix with components as:

$$(\nabla_R L)_{pg} \equiv \frac{\partial L}{\partial R_{pg}}$$

Note that $\tilde{A} = AR$

We can express L as elements of R more explicitly:

$$L = \sum_{j=1}^k \left\{ \sum_{i=1}^n \left(\sum_{r=1}^k A_{ir} R_{rj} \right)^4 - \frac{1}{n} \left[\sum_{i=1}^n \left(\sum_{r=1}^k A_{ir} R_{rj} \right)^2 \right]^2 \right\}$$

$$\frac{\partial L}{\partial R_{pg}} = (\text{Term 1}) - \frac{1}{n} (\text{Term 2})$$

Let's compute these two terms separately.

$$\begin{aligned} \text{Term 1} &= \frac{\partial}{\partial R_{pg}} \sum_{j=1}^k \left[\sum_{i=1}^n \left(\sum_{r=1}^k A_{ir} R_{rj} \right)^4 \right] \\ &= \sum_{j=1}^k \left[\frac{\partial}{\partial R_{pg}} \sum_{i=1}^n \left(\sum_{r=1}^k A_{ir} R_{rj} \right)^4 \right] \quad \text{chain rule.} \\ &= \sum_{j=1}^k \left\{ \sum_{i=1}^n \left[4 \left(\sum_{r=1}^k A_{ir} R_{rj} \right)^3 \cdot \sum_{r=1}^k A_{ir} \frac{\partial R_{rj}}{\partial R_{pg}} \right] \right\} \end{aligned}$$

$$\begin{aligned}
&= \sum_{\bar{j}=1}^k \left\{ \sum_{\bar{i}=1}^n \left[4(\tilde{A}_{\bar{i}\bar{j}})^3 \cdot \sum_{r=1}^k A_{ir} \delta_{rp} \delta_{\bar{j}\bar{q}} \right] \right\} \\
&\quad \downarrow \text{Kronecker delta.} \\
&= \sum_{\bar{j}=1}^k \left\{ \sum_{\bar{i}=1}^n \left[4(\tilde{A}_{\bar{i}\bar{j}})^3 \cdot A_{ip} \delta_{\bar{j}\bar{q}} \right] \right\} \\
&= 4 \sum_{\bar{i}=1}^n (\tilde{A}_{i\bar{q}})^3 A_{ip} = 4 \sum_{\bar{i}=1}^n (A^T)_{pi} (\tilde{A}_{i\bar{q}})^3 \\
&= 4 \cdot (A^T \tilde{A}^{3(\text{ele})})_{p\bar{q}} \\
&\quad \sim 3(\text{ele}) \\
&\quad A \quad \text{means element wise cubed}
\end{aligned}$$

$$\begin{aligned}
\text{Term 2.} &= \frac{\partial}{\partial R_{p\bar{q}}} \sum_{\bar{j}=1}^k \left[\sum_{\bar{i}=1}^n \left(\sum_{r=1}^k A_{ir} R_{r\bar{j}} \right)^2 \right]^2 \\
&= \sum_{\bar{j}=1}^k \left\{ 2 \left[\sum_{\bar{i}=1}^n (\tilde{A}_{i\bar{j}})^2 \right] \cdot \left[\sum_{\bar{i}=1}^n \frac{\partial}{\partial R_{p\bar{q}}} \left(\sum_{r=1}^k A_{ir} R_{r\bar{j}} \right)^2 \right] \right\} \\
&= \sum_{\bar{j}=1}^k \left\{ 2 \left[\sum_{\bar{i}=1}^n (\tilde{A}_{i\bar{j}})^2 \right] \cdot \left[\sum_{\bar{i}=1}^n 2 \left(\sum_{r=1}^k A_{ir} R_{r\bar{j}} \right) \sum_{r=1}^k A_{ir} \delta_{rp} \delta_{\bar{q}\bar{j}} \right] \right\} \\
&= 4 \sum_{\bar{j}=1}^k \left\{ \left[\sum_{\bar{i}=1}^n (\tilde{A}_{i\bar{j}})^2 \right] \cdot \left[\sum_{\bar{i}=1}^n (\tilde{A}_{i\bar{j}}) A_{ip} \delta_{\bar{q}\bar{j}} \right] \right\}
\end{aligned}$$

$$= 4 \left(\sum_{i=1}^n (\hat{A}_{ig})^2 \right) \left(\sum_{i=1}^n (\tilde{A}_{ig}) (A_{ip}) \right)$$

$$= 4 \left(\sum_{i=1}^n (\hat{A}_{ig})^2 \right) (A^T \hat{A})_{pg}$$

Denote $G_g = \sum_{i=1}^n (\hat{A}_{ig})^2 = \sum_{i=1}^n (\tilde{A}_{gi})^T (\tilde{A}_{ig})$

$$= (\tilde{A}^T \tilde{A})_{gg}$$

Which means G_g is the diagonal terms in $(\tilde{A}^T \tilde{A})$

Hence

$$\text{Term 2} = 4 (A^T \tilde{A}) \cdot \text{diag}(\tilde{A}^T \tilde{A})$$

Finally, we get the matrix representation of $P(A; R)$

$$(\nabla_R L)_{pg} = 4 \left[A^T \tilde{A}^{3(\text{ele})} - \frac{1}{n} (A^T \tilde{A}) \text{diag}(\tilde{A}^T \tilde{A}) \right]_{pg}$$



$$\nabla_R L = 4 A^T \left[\tilde{A}^{3(\text{ele})} - \frac{1}{n} \tilde{A} \text{diag}(\tilde{A}^T \tilde{A}) \right]$$

Back to question 1

By Taylor's expansion, we approximate the objective function L at some starting matrix R^0 as:

$$L(R) \simeq L(R^0) + \sum_{p,q} \left. \frac{\partial L}{\partial R_{pq}} \right|_{R^0} (R_{pq} - R_{pq}^0)$$

Since R^0 is a fixed quantity, maximizing $L(R)$ is equivalent to maximizing:

$$\begin{aligned} Q(R) &= \sum_{p,q} \left. \frac{\partial L}{\partial R_{pq}} \right|_{R^0} R_{pq} \\ &= \sum_{p,q} [P(R^0)]_{pq} R_{pq} \\ &= \text{trace}(R^T \cdot P_0) \end{aligned}$$

Where I replaced $P(R^0)$ as P_0 to indicate it's a constant with respect to the maximizing procedure. So the problem reduces to:

$$\text{Argmax}_R \left[\text{trace}(R^T P_0) \right] ; R^T R = I$$

Consider the svd of P_0

$$P_0 = U_0 \Sigma_0 V_0^T$$



$$\text{trace}(R^T P_0) = \text{trace}(R^T U_0 \Sigma_0 V_0^T)$$

$$= \text{trace}(V_0^T R^T U_0 \Sigma_0) \quad (\text{cyclic property of trace})$$

$$= \text{trace}(Q^T \Sigma_0) ; Q = U_0^T R V_0$$

Note that the matrix Q is an orthonormal matrix

$$Q^T Q = (V_0^T R^T U_0)(U_0^T R V_0)$$

$$= V_0^T R^T R V_0 = V_0^T V_0 = I$$

Since Σ_0 is a diagonal matrix, the trace can be written as

$$\text{trace}(Q^T \Sigma_0) = \sum_i \sigma_i \boxed{Q_{ii}} \leq \sum_i \sigma_i$$



For orthogonal matrix, $|Q_{ii}| \leq 1$

Equality holds when $Q = I$

In other words,

$$Q = U_0^T R V_0 = I \longrightarrow R = U_0 V_0^T$$

And this answers the question on why do we do svd on P matrix in order to get the best rotation matrix R in the sense of varimax.

Afterword

In fact we can still use gradient descent algorithm like:

$$\begin{aligned}\hat{R}_{i+1} &= R_i + \eta \cdot P(R_i; A) \quad \{ \eta : \text{learning rate} \} \\ \hat{R}_{i+1} &= U_{i+1} \Sigma_{i+1} V_{i+1}^T \quad \{ \text{do svd on } \hat{R}_{i+1} \} \\ R_{i+1} &= U_{i+1} V_{i+1}^T \quad \{ \text{force } R_{i+1} \text{ to be orthonormal} \} \\ \text{monitor } L(R_{i+1}; A) &\text{ to break loop.}\end{aligned}$$

I have coded this algorithm in the "learning rate version" branch in GitHub repo of this assignment.

After testing various learning rates, I have found that for

small learning rate, the results may differ from the direct svd algorithm, however for large learning rates, the results match. This happens because when learning rate is large, the effect of P becomes dominant, which reduces to the direct svd method.